

Module 2: Recommender Systems

Introductory lecture

`fredrik.johansson@chalmers.se`

January 23, 2024

Recommender systems

- **Recommender systems** are used all over the internet to serve ads, suggest purchases, rank posts in social feeds, ...



NETFLIX

amazon.com

Linked in

Spotify

¹Image from RealPython.com

Recommender systems

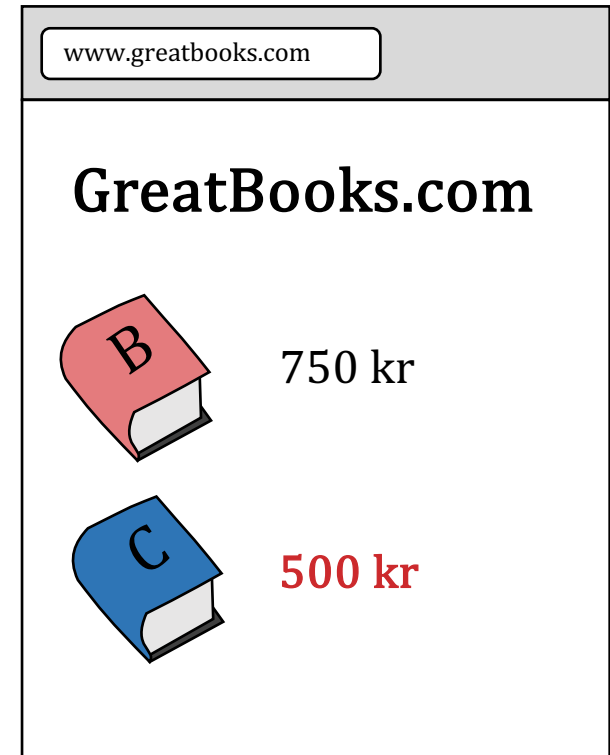
- Famously, **Netflix** claimed in 2015 that their recommender system was worth \$1 billion per year¹
- They were early in the game—started with ratings and recommendations of mail-order movies

NETFLIX

¹Gomez-Urbe & Hunt, *TMIS*, 2015

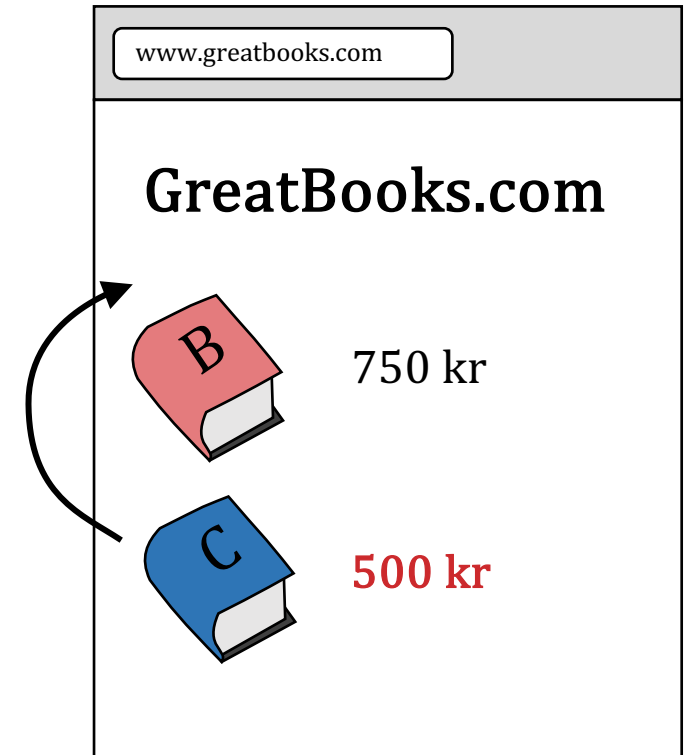
A first example

- Imagine you're running a book **web store**
- You have catalogue of 100 000 books which you need to **display** to users
- Which books should you display in order to **maximize sales**?



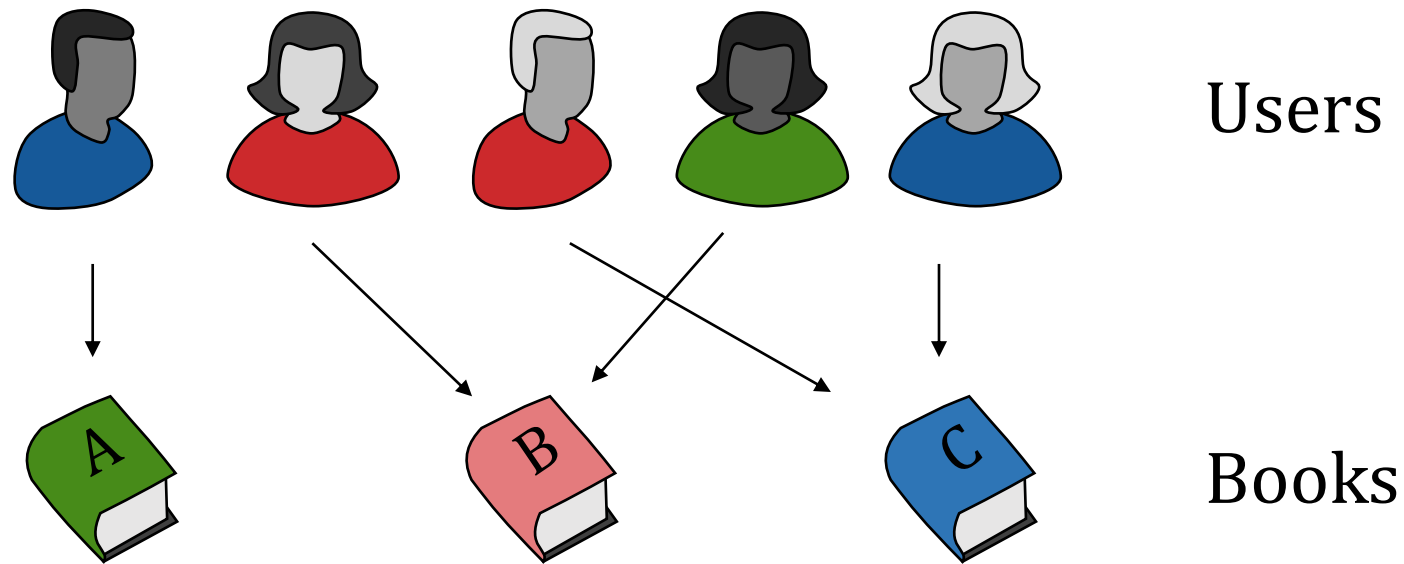
A uniform strategy

- A traditional strategy is to display books that sell well overall or **books on sale**
- **On average**, these books are likely to sell—but not to every user
- Can we do **better**?



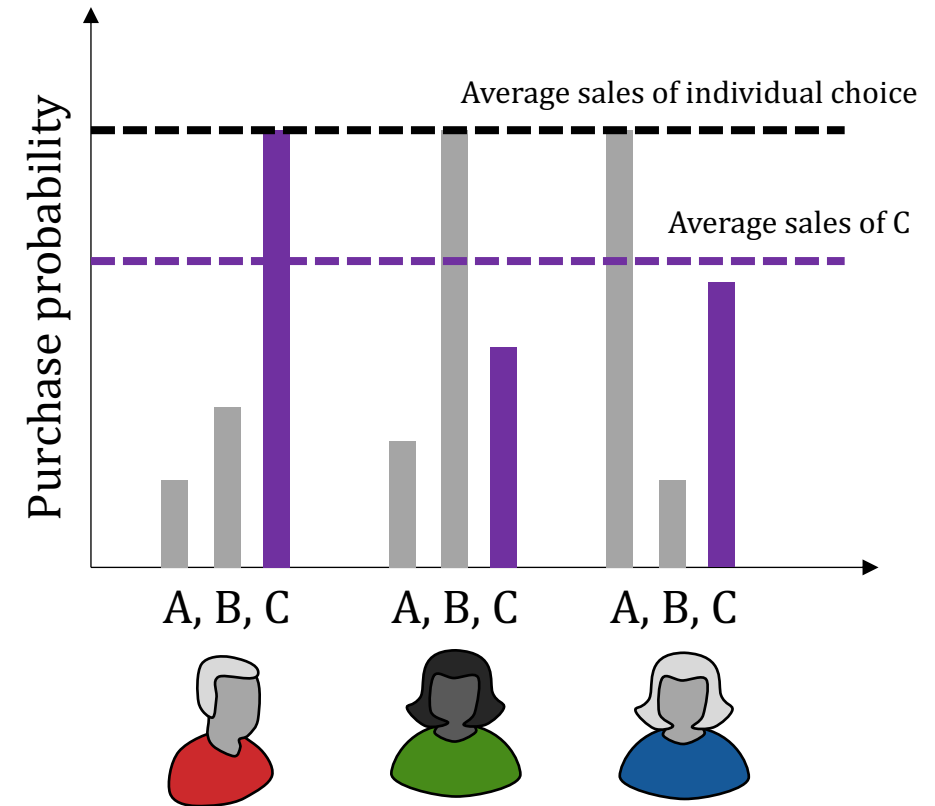
A personalized policy

- Recommender systems try to **personalize** the display policy
- Each user is assigned **their own** recommendations



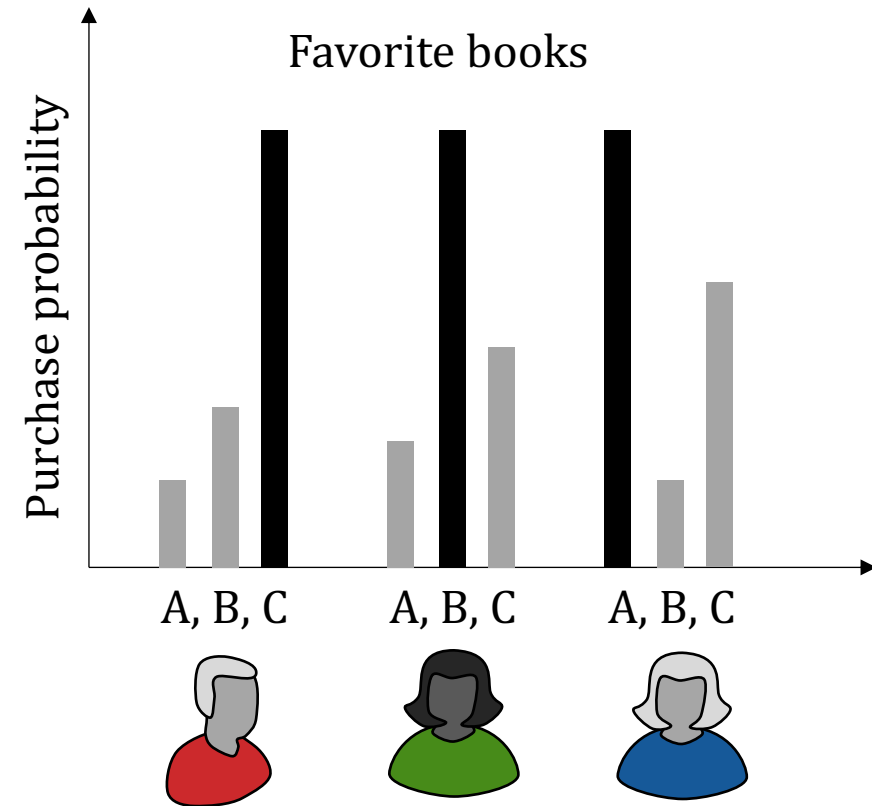
Why is individual preferable?

- Say that every person has **at least one book** they would buy with 60% probability
- The average sales of any single book may be much lower than 60%



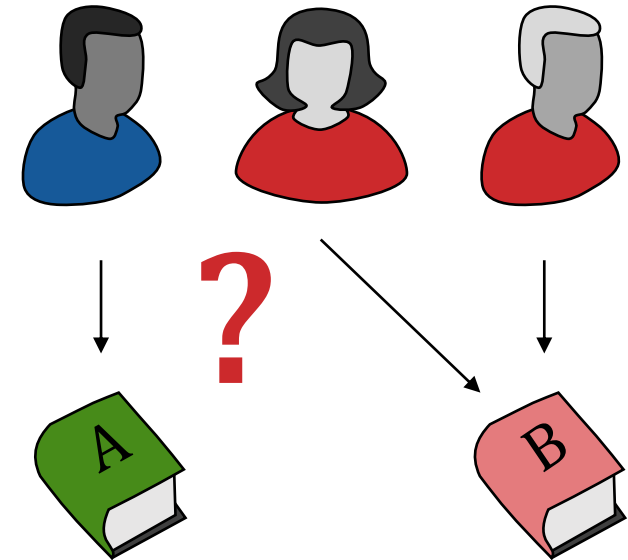
Why is individual preferable?

- Say that every person has **at least one book** they would buy with 60% probability
- How do we figure out users' favorite books?



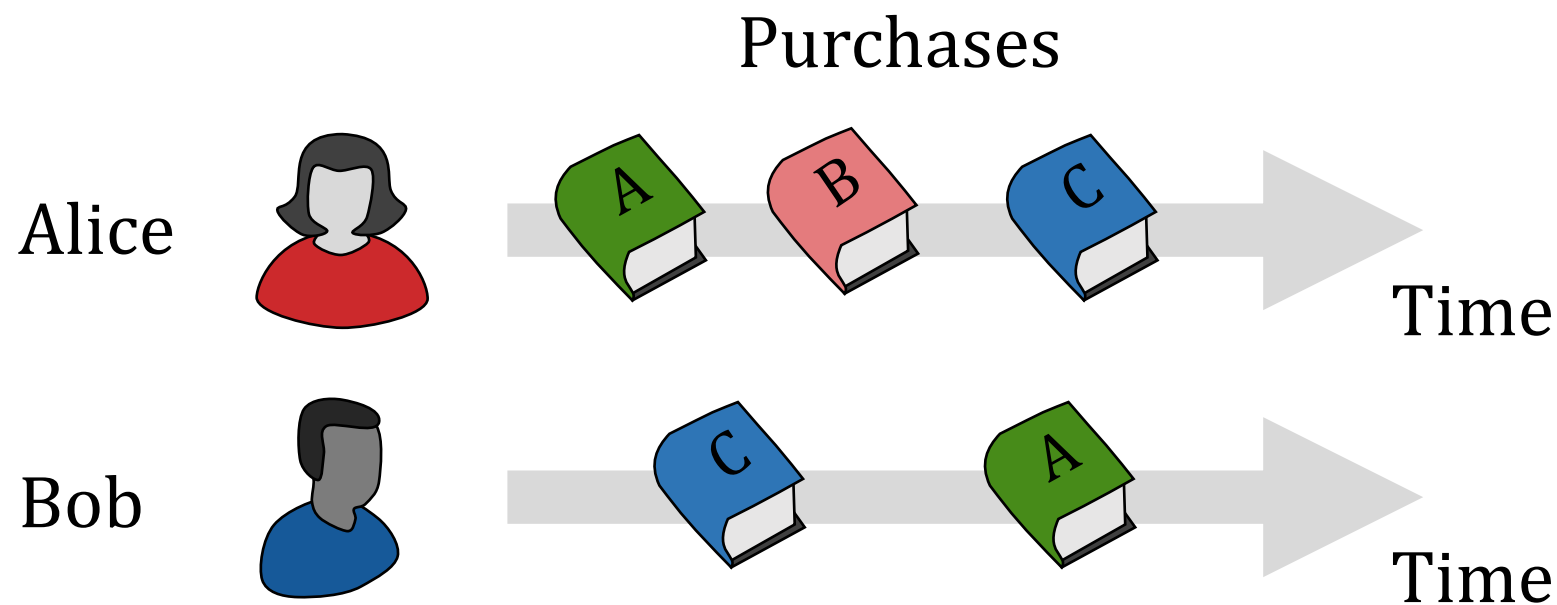
Models of user preference

- To **distinguish** users, we must know something about them
- We **could ask them** what they like, but this is not always a good indicator*
- Instead, we can **collect data passively**



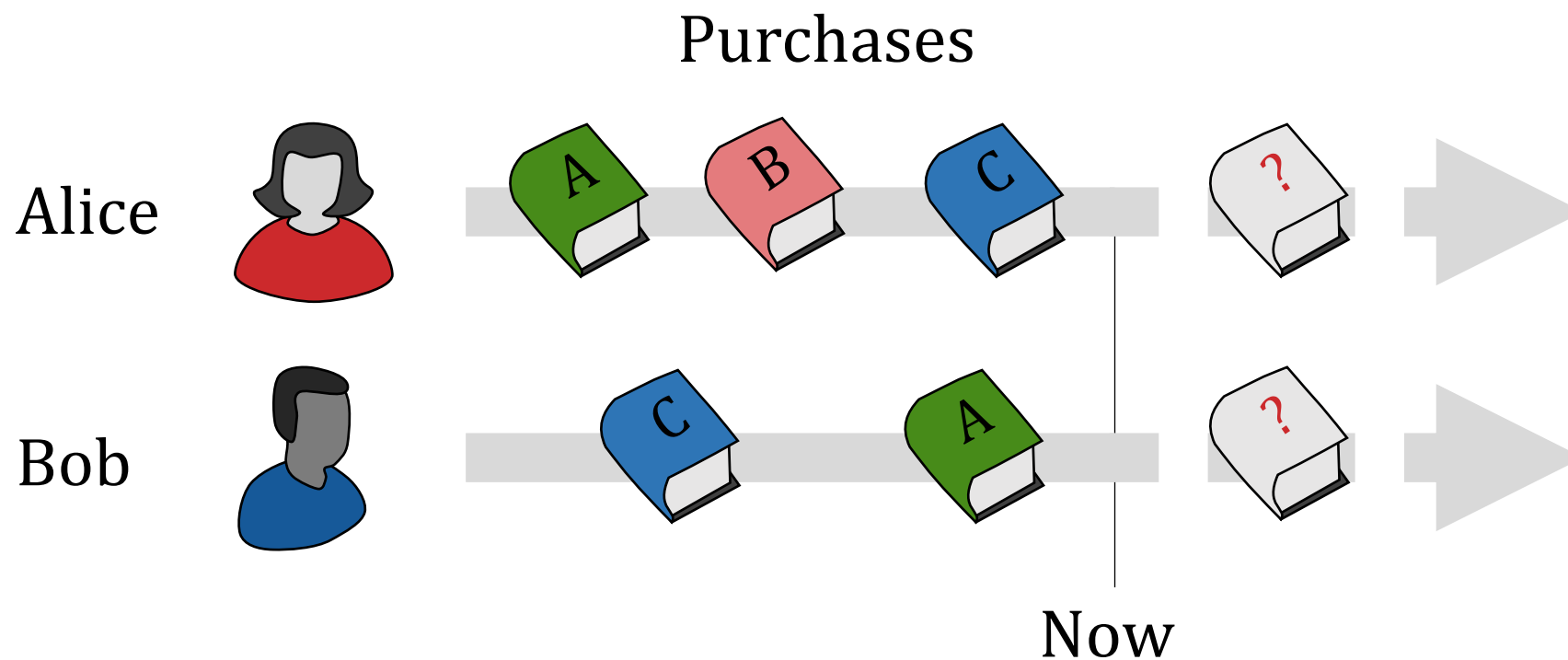
Purchase logs

- It is common for companies to store **purchase logs**



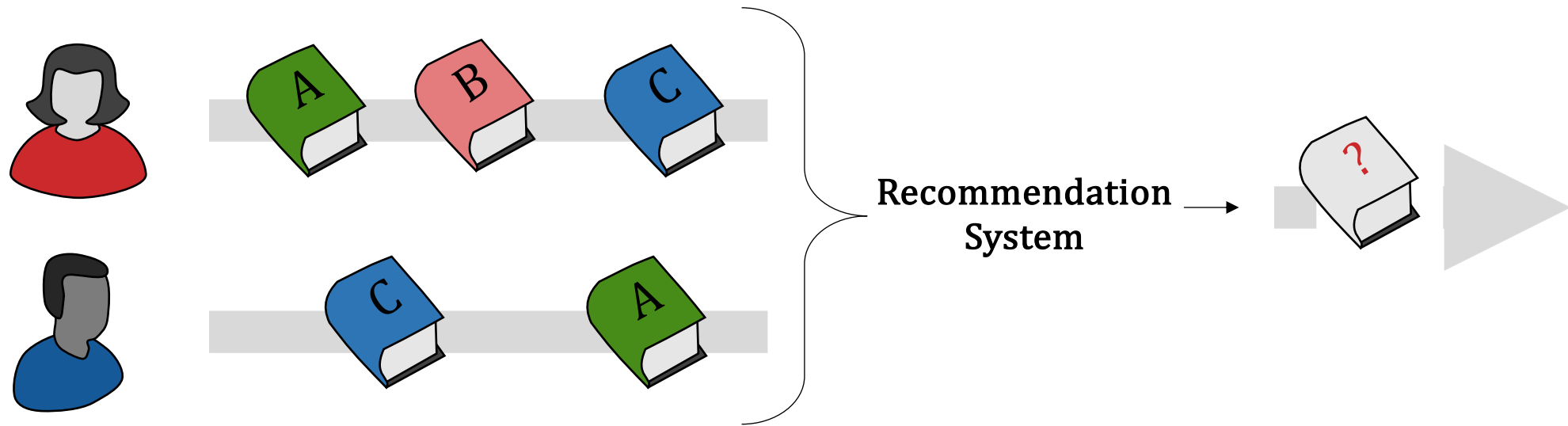
Predicting future purchases

- Based on these logs, we can try to **predict** future purchases



Recommendation systems

- **Recommendation systems** attempt to solve this problem



Outline: Recommendation systems

1. Objectives & proxies
2. Learning to recommend
3. Recommenders in practice
4. Evaluation and deployment

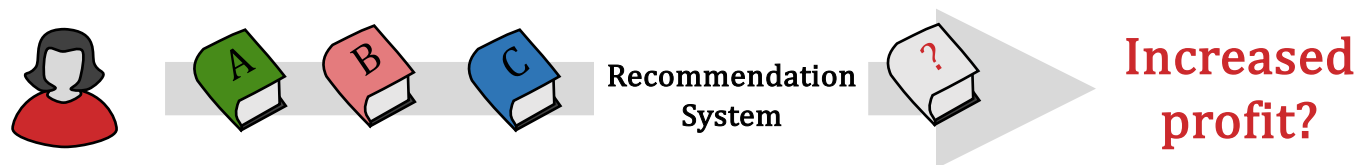
Objectives & Proxies

Objectives: What do we want?

- An **objective** should encapsulate **what we want** from our system
- It should be the **subject of evaluation**—how well are we doing?

- **Examples:**

- Profit
- Number of users
- User engagement
- User satisfaction



Delayed objectives

- Long-term (or “delayed”) objectives are **hard to optimize directly**
- 1. Not always easy to **specify**—even in natural language
 - **Example:** How do we define “engaged”?
- 2. Often hard to **measure**
 - **Example:** Surveying user satisfaction is labor intensive
- 3. Even harder to **optimize**. What was the cause?
 - **Example:** Profit determined by many things

Short reflective assignment

1. Reflect, take notes (2 min)

For the bookstore example,

- Pick an objective
- How would you measure it?
- How would you optimize it?

Example: Movie-streaming service

- **Objective:** Maximize profit
- We can measure profit. Great!
 - But what is the **effect** of recommendations on profit...?

Example: Movie-streaming service

- **Objective:** Maximize profit
- We can measure profit. Great!
 - But what is the effect of recommendations on profit...?
- Profit comes from subscriptions. Great!
 - But what is the **effect** on subscriptions?

Example: Movie-streaming service

- **Objective:** Maximize profit
- We can measure profit. Great!
 - But what is the effect of recommendations on profit...?
- Profit comes from subscriptions. Great!
 - But what is the effect on subscriptions?
- Subscriptions may be related to user engagement. Great...?

Example: Movie-streaming service

- **Objective:** Maximize profit

- We can measure profit. Great!
 - But what is the effect of recommendations on profit...?
 - Profit comes from subscriptions. Great!
 - But what is the effect on subscriptions?
 - Subscriptions may be related to user engagement. Great...?
- 
- The diagram consists of two curved arrows pointing from the left towards the list items. The top arrow is labeled 'Proxy!' and points from the first item 'We can measure profit. Great!' to the second item 'Profit comes from subscriptions. Great!'. The bottom arrow is also labeled 'Proxy!' and points from the second item to the third item 'Subscriptions may be related to user engagement. Great...?'.

Proxies: What can we measure and optimize?

- A common proxy for user engagement is **user ratings**

★★★★☆ ▾ 27,484 ratings | 1000+ answered questions

- **Assumption:** If a user is likely to rate a product highly, they are more likely to engage with it

“ The length of service of subscribers is related to the number of movies they watch and enjoy. If subscribers fail to find movies that interest and engage them, they tend to abandon the service. Connecting subscribers to movies that they will love is therefore critical to both the subscribers and the company. ”

User ratings as proxies



- Ratings are **easy to measure**. We can ask users to give them
- Ratings pertain to a **specific product** that we can recommend
- In contrast, for example, no single movie in Netflix's service determines profit directly

Maximizing user ratings

- **How** can we recommend products that will be highly rated?
- **Abstraction:** First **predict** ratings, then pick highly rated product
- Note that we have abstracted **significantly**:
 - Maximize profit → maximize engagement
→ maximize ratings → maximize predicted ratings

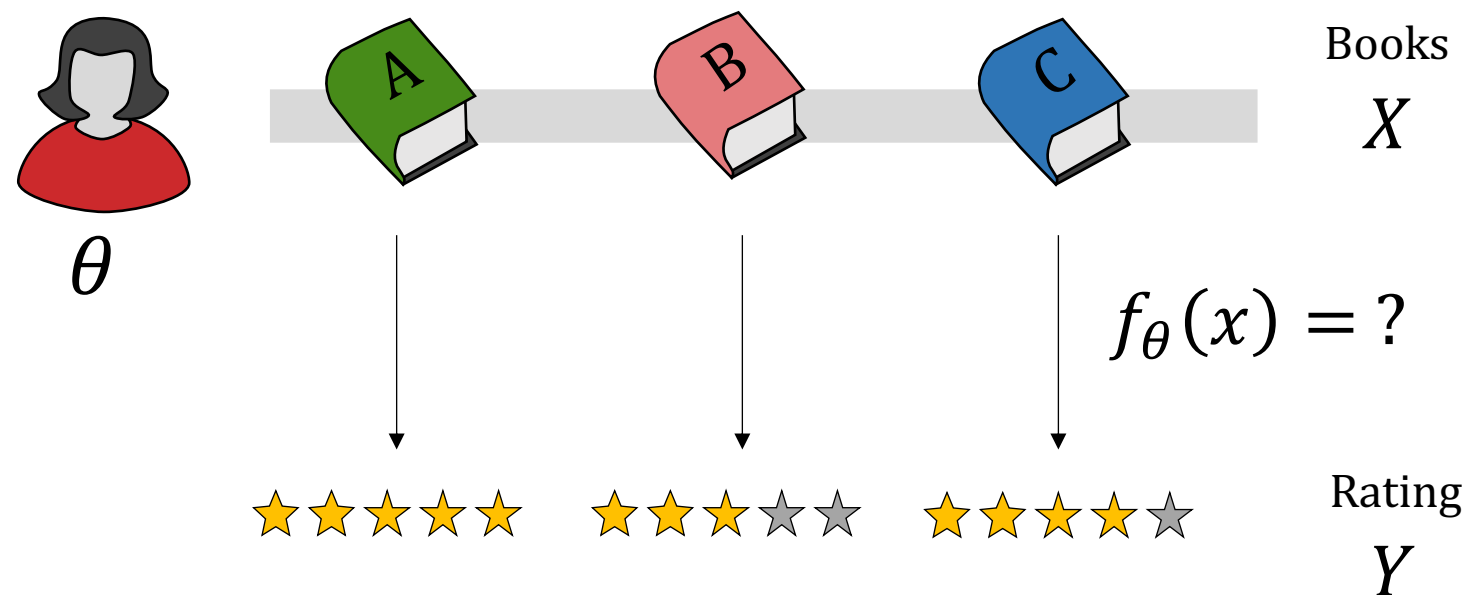
Learning to recommend

Personalized recommendations

- **Baseline:** Recommend highest-rated product overall
- **Personalization** builds on the idea that the best recommendation is not the same for everyone.
- Predicting which product will be highly **rated by a specific user** allows us to personalize recommendations

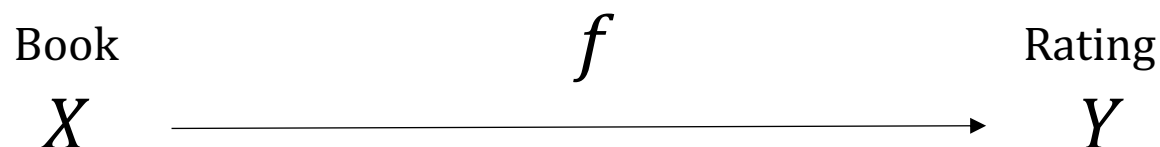
How does AI fit in?

- **Predicting** future events is a classical example of AI
- We look for a personal mapping f between books X and ratings Y



Learning from data

- We don't know the function f when we start



- But we have **records** of purchased books X and ratings Y
- We can use **machine learning** to approximate $f(x)$

Historical data

- **Storing purchases** is trivial in online stores

			
	×		×
	×	×	×
			×
		×	
	×		

Often, we know something **about the products**



Romance



Thriller



Children's book

Formalizing recommenders

- Let's use this example, but now users **give ratings** [0–5]

	x		x
x		x	x
			x
x		x	

y_{ij} = Score for item j by user i

Formalizing recommenders

- Assume we know something **about the books**. Call this X .

$Y =$

		Books (ratings)				
Users		5	5	?	0	0
		5	?	4	0	0
		0	?	0	5	5
		0				?
		Books (features)				
Features		1	1	1	0	0
		0	0	1	1	1
		X_1	X_2	X_3	X_4	X_5

Suspense
Romance

Formalizing recommenders

- We could figure something out about what users might like

$Y =$

		Books				
Users		5	5	?	0	0
		4	?	4	0	0
		0	?	0	5	5
		0	0	?	4	?

$X =$

Features		1	1	1	0	0
		0	0	1	1	1
		X_1	X_2	X_3	X_4	X_5

Suspense
Romance

It seems User 1 likes suspense novels!

Formalizing recommenders

- What if we knew something about **what users like**? Call this Θ .

		Books					$\Theta =$			
							Suspense	Romance		
$Y =$	Users	5	5	?	0	0	5	0	Θ_1	
		4	?	4	0	0	4	0	Θ_2	
		0	?	0	5	5	0	5	Θ_3	
		0	0	?	4	?	0	4	Θ_4	
$X =$	Features	1	1	1	0	0	User preferences			
		0	0	1	1	1				

Formalizing recommenders

- We could figure out which users would like similar things

		Books					$\Theta =$			
							Suspense	Romance		
$Y =$	Users	5	5	?	0	0	5	0	Θ_1	
		4	?	4	0	0	4	0	Θ_2	
		0	?	0	5	5	0	5	Θ_3	
		0	0	?	4	?	0	4	Θ_4	
$X =$	Features	1	1	1	0	0				
		0	0	1	1	1				

Formalizing recommenders

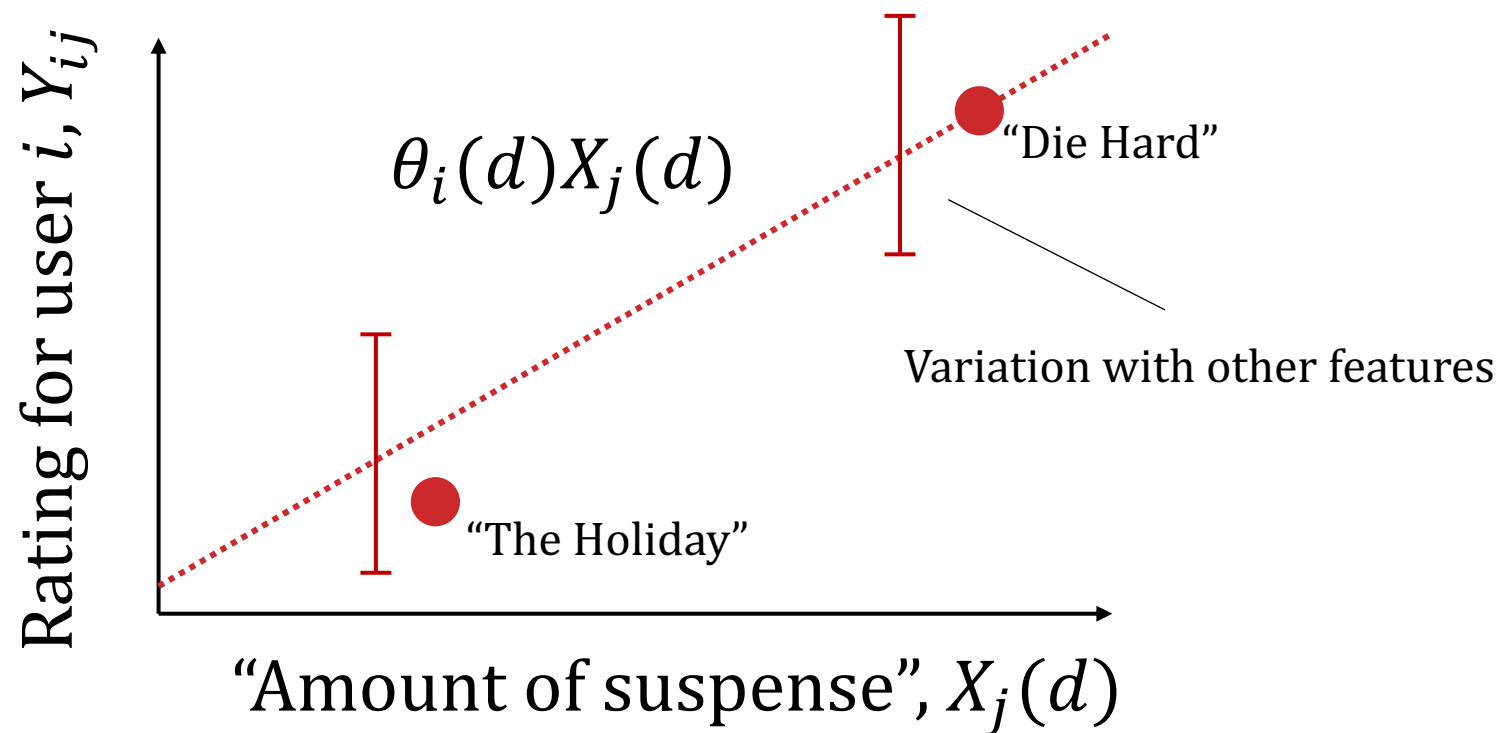
- Most recommender systems* are based on **user ratings** Y , **user preferences** Θ and **product features** X
- Different systems make **different assumptions**
- **A common assumption:** $Y \sim f(X; \Theta)$ has a **particular form**
- For example: $Y = \Theta X$ (matrix multiplication/linear form)

* Sometimes also user history (beyond preferences)

Linear ratings

- **Assume:** Each user has their own linear rating function
- Every book feature, e.g., genre, **contributes linearly** to rating

$$Y_{ij} = \sum_{d=1}^D \theta_i(d) X_j(d)$$



Algorithm 1: Content filtering

- Finding parameters for each user by **minimizing squared error**

The diagram shows the optimization equation for content filtering, with annotations explaining its components:

$$\min_{\theta_1, \dots, \theta_n} \frac{1}{2} \sum_{j=1}^m \sum_{i=1}^n (\theta_i^\top x_j - y_{ij})^2$$

Annotations and their corresponding parts of the equation:

- Squared prediction error**: Points to the term $(\theta_i^\top x_j - y_{ij})^2$.
- Sum over products**: Points to the summation $\sum_{j=1}^m$.
- Product features**: Points to the term x_j .
- User preferences**: Points to the term $\theta_i^\top$.
- Sum over users**: Points to the summation $\sum_{i=1}^n$.

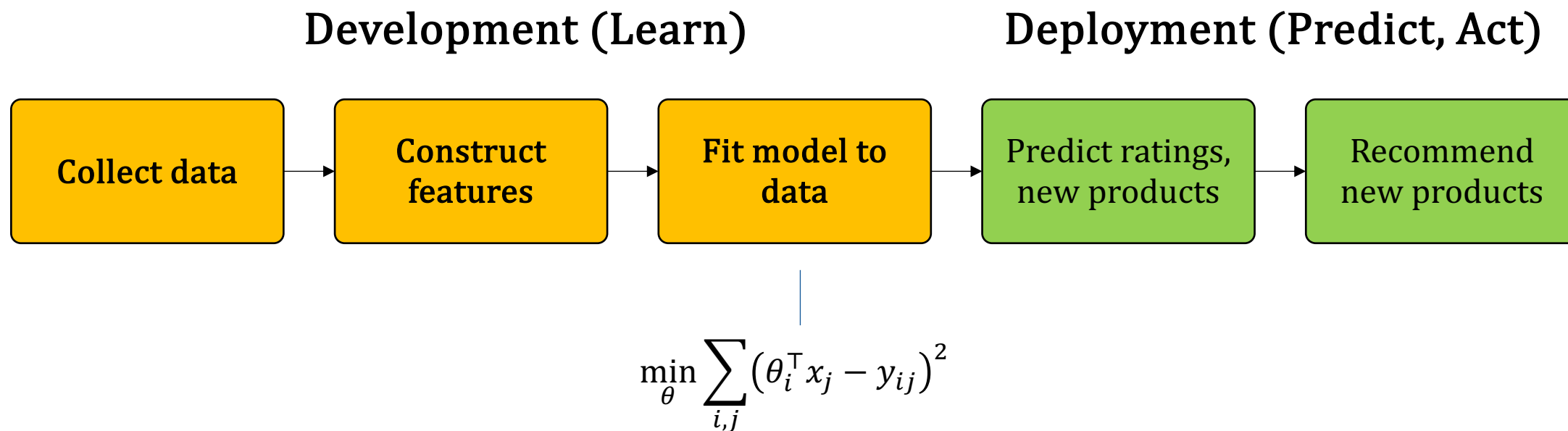
Theme: “Empirical risk minimization”

- Finding model θ that **minimizes error on observed data**

$$\min_{\theta_1, \dots, \theta_n} \frac{1}{2} \sum_{j=1}^m \sum_{i=1}^n (\theta_i^\top x_j - y_{ij})^2$$

- By far the most common AI/ML strategy today

Learn, predict, act (Idealized case)



Problem 1: Incomplete data

- However, most users don't rate all products. Y has **missing data**!

				
	×		×	Y_1
	×	×	×	Y
			×	Y_3
		×		Y_4
	×			Y_5

$$\sum_{j=1}^m \sum_{i=1}^n (\theta_i^\top x_j - y_{ij})^2$$

We don't observe
all the terms in the
two sums

Fitting parameters to observed ratings

Let $r(i, j) = 1$ that user i rated product j and $r(i, j) = 0$ otherwise

We can limit our objective to observed ratings

$$\min_{\theta} \frac{1}{2} \sum_{j=1}^m \sum_{i:r(i,j)=1} (\theta_i x_j - y_{ij})^2 + \lambda$$

Problem 1: Non-uniform missingness*

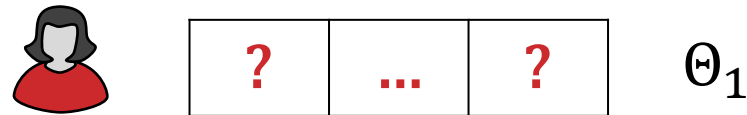
- Worse, in real data, **most of Y** is missing—and **not uniformly!**

	A	B											C				
User 1	X													X			
User 2	X		X										X				
User 3																	
User 4			X														
User 5	X																

* More on this next week!

Problem 2: High dimensionality

- We often have more products than ratings. What about features?



- Separate linear models for each user **will overfit** if we have more **features** than ratings for that user

Regularization

- We can mitigate overfitting by adding regularization

$$\min_{\theta} \quad \frac{1}{2} \sum_{j=1}^m \sum_{i:r(i,j)=1} (\theta_i x_j - y_{ij})^2 + \lambda \sum_{i=1}^m \theta_i^2$$

- The example above is “Ridge regression”*
- Favors models with *small* parameters θ

Regularization

* Also called Tikhonov regularization

Limitations of content filtering

- A drawback of content filtering is that it relies completely on having **good product features X**
- We also **need enough ratings** for a user i to begin fitting θ_i
- Even if we do have such product features, separate regressions ignore the fact that many users **behave similarly**

An alternative to content filtering

Content filtering

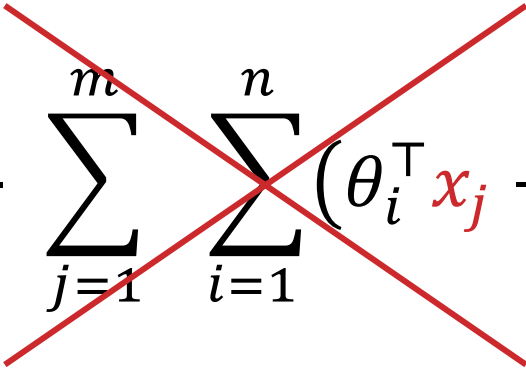
- **User-product** and **product-feature** data
- “**Similar products** get similar scores from users”

Collaborative filtering

- **User-product** data only
- “**Users with similar histories** give similar scores to the same products”

Algorithm 2: Collaborative filtering

- What if we **don't have** product features X . Can we still predict Y ?
- We can't do linear regression anymore

$$\min_{\theta} \frac{1}{2} \sum_{j=1}^m \sum_{i=1}^n (\theta_i^T \mathbf{x}_j - y_{ij})^2$$


Collaborative filtering

- **User-product** data only
- “**Similar users** give similar scores to products”

Matrix decomposition

- Think about the matrix Y as a product ΘX where
 - Θ has many (n) rows (users), but few (d) columns (features)
 - X have few (d) rows (features), but many (m) columns (books)
- We optimize over Θ and X together—**matrix decomposition**

$$\min_{\Theta \in \mathbb{R}^{n \times d}, X \in \mathbb{R}^{d \times m}} \frac{1}{2} \sum_{j=1}^m \sum_{i:r(i,j)=1} (\theta_i^\top x_j - y_{ij})^2 + \dots$$

* Typically, regularization will be used here as well

Matrix decomposition

		Books					$\Theta =$		
$Y =$	Users	5	5		0	0	?	?	Θ_1
		4		4	0	0	?	?	Θ_2
		0		0	5	5	?	?	Θ_3
		0	0		4		?	?	Θ_4
$X =$	Features	?	?	?	?	?	}		No fixed meaning anymore
		?	?	?	?	?			

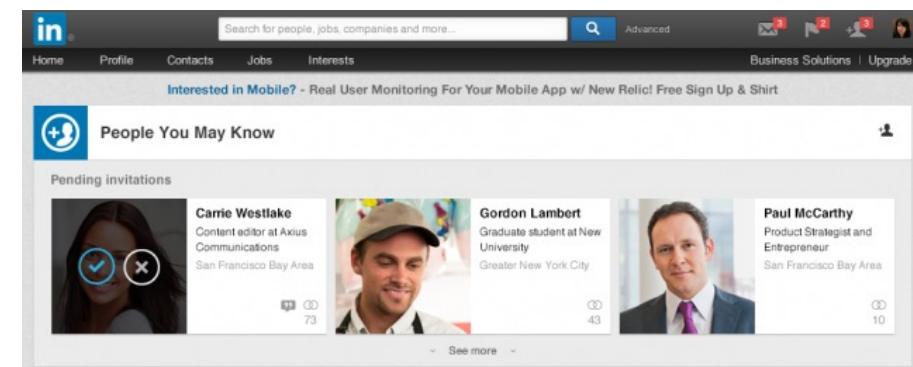
Collaborative filtering

- - Matrix decomposition** variants have been hugely successful in collaborative filtering recommenders
- Can be combined with **regularization** as before, or by limiting the rank (dimension) of the decomposition parts Θ, X ,
- ... or be combined with **content-based models**

Case studies: Recommenders in practice

Case study 1: LinkedIn¹

- **Several** recommendation problems:
 - Job recommendations, **people-you-may-know**, LinkedIn feed, candidates, courses, and many more
- Value proposition for **users**:
 - “Connect, Stay informed, Get hired”
- Value for **customers** (companies):
 - “Hire, market, sell, work”



¹Slides from Deepak Agarwal, <https://www.youtube.com/watch?v=VJOtr47V0eo>

Objectives & proxies

- **Long-term objectives**

Return visits to site, user connections, job applications

- **Short-term proxies**

Click-through events, connection probability, application probability

- “Optimize proxies with a short feedback loop”

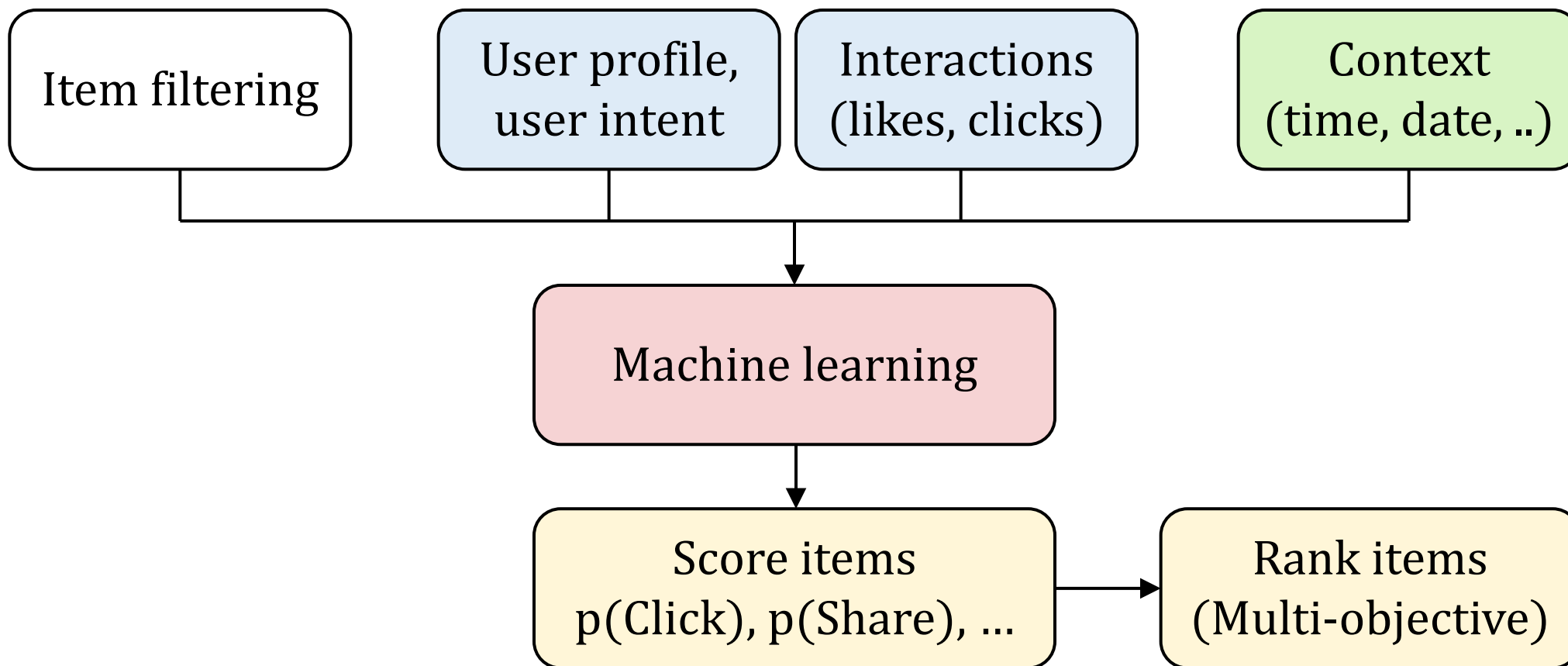
¹Slides from Deepak Agarwal, <https://www.youtube.com/watch?v=VJOtr47V0eo>

Types of data

- **User profiles:** Supplied by the user (user has incentive)
- **User history:** Click, search, interaction information
- **Item features:** Features of articles, posts, listings
- This covers both user-product, product-feature and other kinds of data. We can build rich models!

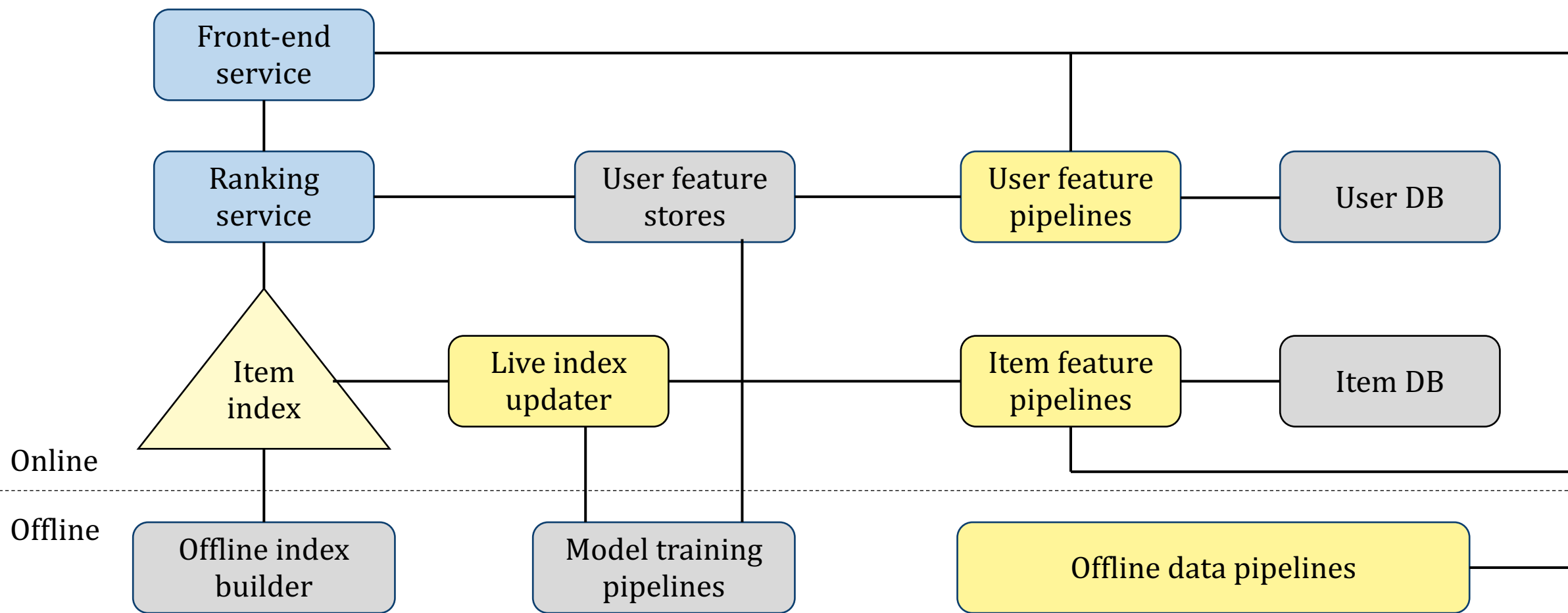
¹Slides from Deepak Agarwal, <https://www.youtube.com/watch?v=VJOtr47V0eo>

System overview



¹Slides from Deepak Agarwal, <https://www.youtube.com/watch?v=VJOtr47V0eo>

System architecture*



*The details are not important—the context of the model is. ¹Slides from Deepak Agarwal

Delivery mechanisms—How are recommendations given?

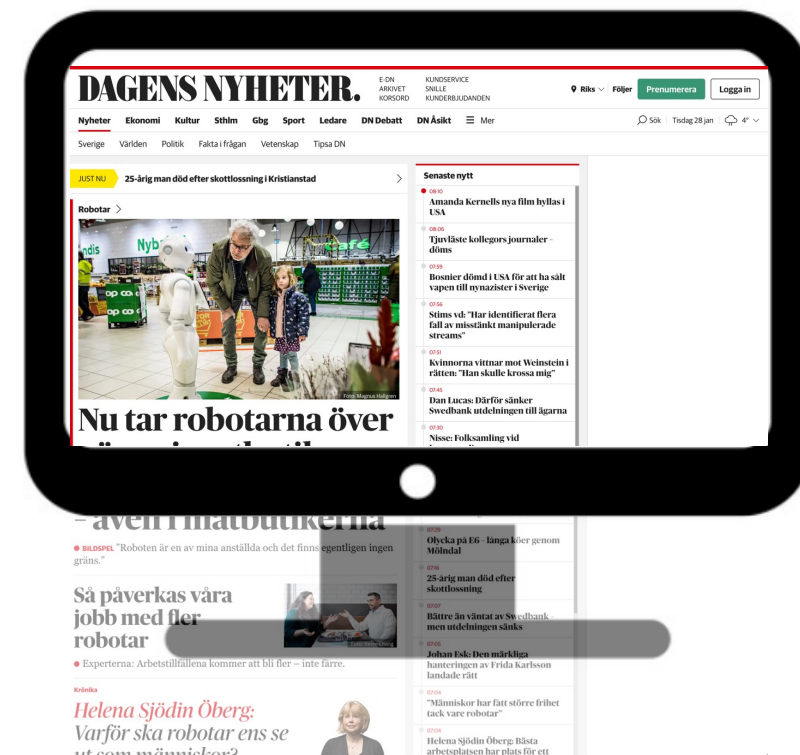
- **Pull model:** Initiated by users
Example: When users visit the website, use the app
- **Push model:** Initiated by system
Example: Send emails, notifications, etc
- Push model historically a big “false-positive” problem!

¹Slides from Deepak Agarwal, <https://www.youtube.com/watch?v=VJOtr47V0eo>

The value of data tracking

- The right **user data** is often more important than the right model
- An important nuance: Instead of tracking what the user clicked among *all ads* served to a page, tracking **what was visible**
- This may have a larger impact than many model updates

Visible to the user



Invisible to the user

The value of choosing the right metric

- A **click-through event** is a short-term (very noisy) proxy for revenue or engagement
- Clicks are directly related to products, e.g., job ads
- However, for LinkedIn, **shares and comments** are more valuable in the long-term as they increase **user engagement**

Content inventory

- It's easy to get caught up in modeling, but don't forget **what** you are recommending to the user—what are your products?
- If you recommend the best item **among a set of bad items**, ...



(This tweet is a few years old)

Unintended side-effects

- Netflix were criticized for “racially targeting” **ads/thumbnails**
- Personalization can mean that groups are **treated differently**
- Side-effects such as these are often **at odds** with chosen objectives

¹Image: theguardian.com

Netflix - treat your black customers with the respect they deserve

Lanre Bakare



By promoting their shows with bit-part black actors, Netflix is distorting the truth about Hollywood diversity



▲ 'Racially targeted' promotional images for the Netflix film Like Father. Photograph: Twitter Kelly Quantrill @codetrill and stacia l. brown @slb79

Conclusions

- Recommendation problems are **everywhere**
- The "**predict ratings**" abstraction is so common that recommender systems is often described this way
 - The literature on predicting user ratings is huge
 - but it is not all that matters!
- Remember that your algorithm should **fit your objective!**
 - Evaluation should also be done w.r.t. the objective

Evaluation & deployment (Follow-up session)